Project Name: PathMate.ai

Team Members:

- **Dohyeon Lee** (dohyeon.lee@stonybrook.edu)
- **Heesang Kim** (heesang.kim@stonybrook.edu)

1. Introduction

1.1 Product Description

PathMate.ai is an AI-driven career management platform that integrates fragmented job preparation into one. Apart from traditional simple job posting arrangements, it deeply analyzes users' profiles (GPA, major, project, skill stack, etc.) to match optimal jobs, and provides data-driven support for the entire workflow from document preparation to interview practice.

1.2 Scope

Core Features:

- Provides a career dashboard based on user competency data
- Job analysis and personalized matching scores (Fit Score)
- Create and calibrate customized cover letter drafts using AI
- Hiring Simulator and Feedback Report for Practice AI

1.3 Target Users

Graduating Seniors
Students entering the job market who need immediate, realistic matching for their first professional roles.

Active Job Seekers
Individuals facing difficulties in the current employment market seeking a competitive edge.

Early-Career Planners
High school or early university students looking to build a strategic roadmap for their future career paths.

2. Requirements

Since the previous SRS phase and UI Mockup announcement, we have redefined the requirements around the following four key functions.

2.1 Functional Requirements

FR-1: The system should display a 'Readiness Score' on the main screen, which scores the user's current readiness.

FR-2: Based on the activity history of the user, the 'Weekly Tasks' to be completed this week and the full career roadmap should be visualized and provided.

FR-3: A 'Fit Score' should be calculated by comparing the user's profile with the job posting.

FR-4: Not only the score, but also the 'Match Logic' and organizational culture suitability and pay indicators of why this job was matched should be detailed.

FR-5: Custom interview questions should be generated based on selected job postings and user profiles.

FR-6: Multiple feedback reports such as Structure, Impact, and Clarity should be provided by analyzing the user's answers.

FR-7: Create a draft cover letter linking the key keywords in the job description with the user's experience.

FR-8: Users should be informed of the lack of job-related evidence in the prepared document through the 'Missing Evidence Flag' and encouraged to supplement it.

2.2 Non-functional Requirements

Performance: Key analysis results and AI responses must be completed within 15 seconds.

Security: Provides secure authentication with Google OAuth, and encrypts and manages personal competency data.

Usability: Minimize language barriers by providing Korean and English UI switching functions.

3. System Architecture

3.1 Overview

(1) Overview

Based on the **Client-Server architecture** based on **Next.js 16.1.6**, the system provides efficient data and authentication management through **Supabase (PostgreSQL 16)** and advanced AI services through **OpenAI (GPT-4o)** interworking. In particular, by tightly combining design and business logic using **Tailwind CSS 4.0** and Server Actions, it has a full-stack structure that ensures high scalability and development efficiency without the need for separate server infrastructure operations.

(2) Third-party Components & Frameworks

Key third-party technologies utilize **OpenAI SDK (v6.29.0)** to handle core AI matching and question generation logic, and implement intuitive UI icons with **Lucide React (v0.5777.0)**. In addition, clsx and tailwind-merge are combined to provide flexibility in styling and maximize the maintenance of code.

(3) Design Patterns

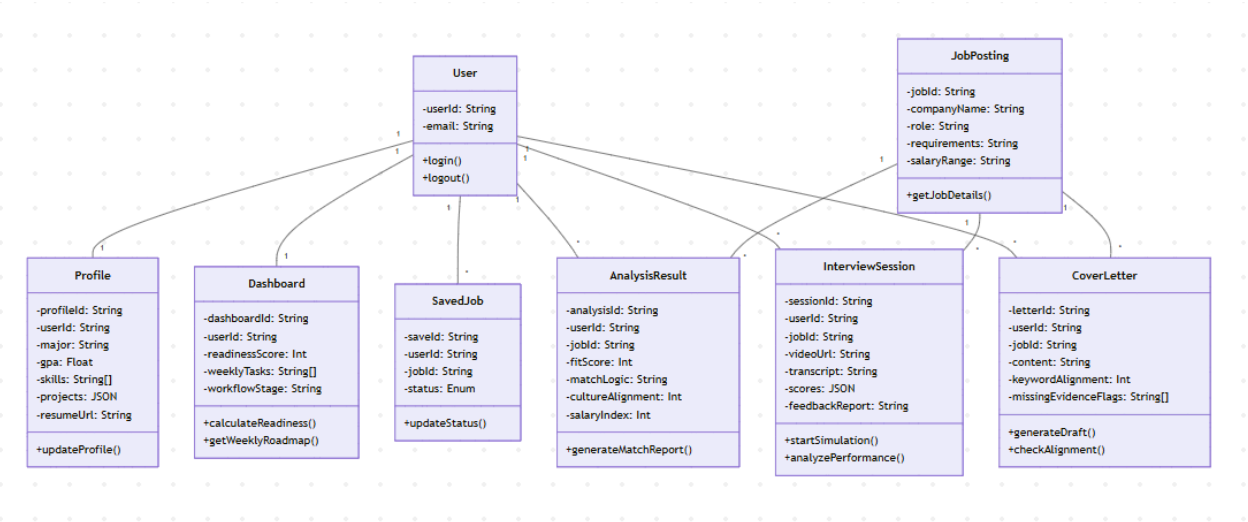
In terms of software design, MVC patterns are applied to clearly separate and manage data (Supabase), UI (React), and logic (Server Actions), and UI is designed in reusable units through a component-based architecture to increase development efficiency and code quality at the same time.

(4) Rationale

The integrated stack based on Next.js and TypeScript accelerates development, and utilizes Vercel and Supabase to create a cost-effective, high-performance infrastructure. By combining OpenAI APIs, we maximize the accuracy of matching and feedback, which is the core of the service.

3.2 Data Design

This UML class diagram outlines the core data structure behind PathMate.ai. These classes basically act as the bridge between the database (Supabase) and the Next.js frontend.



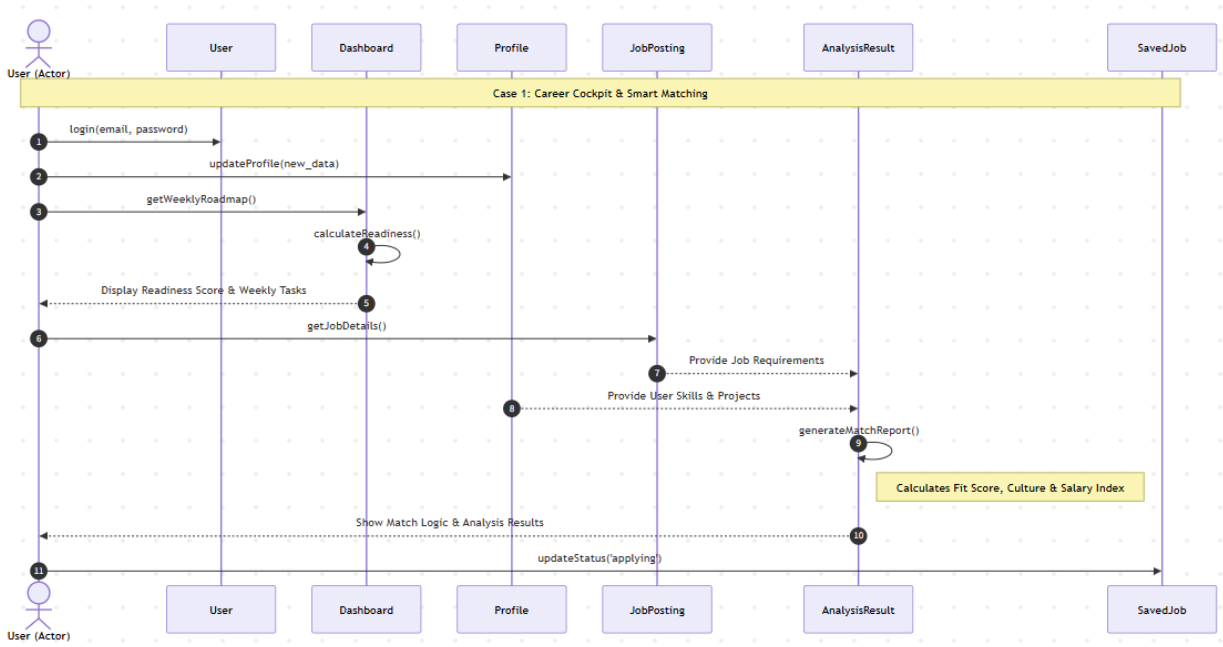
UML Class Diagram

Dictionary

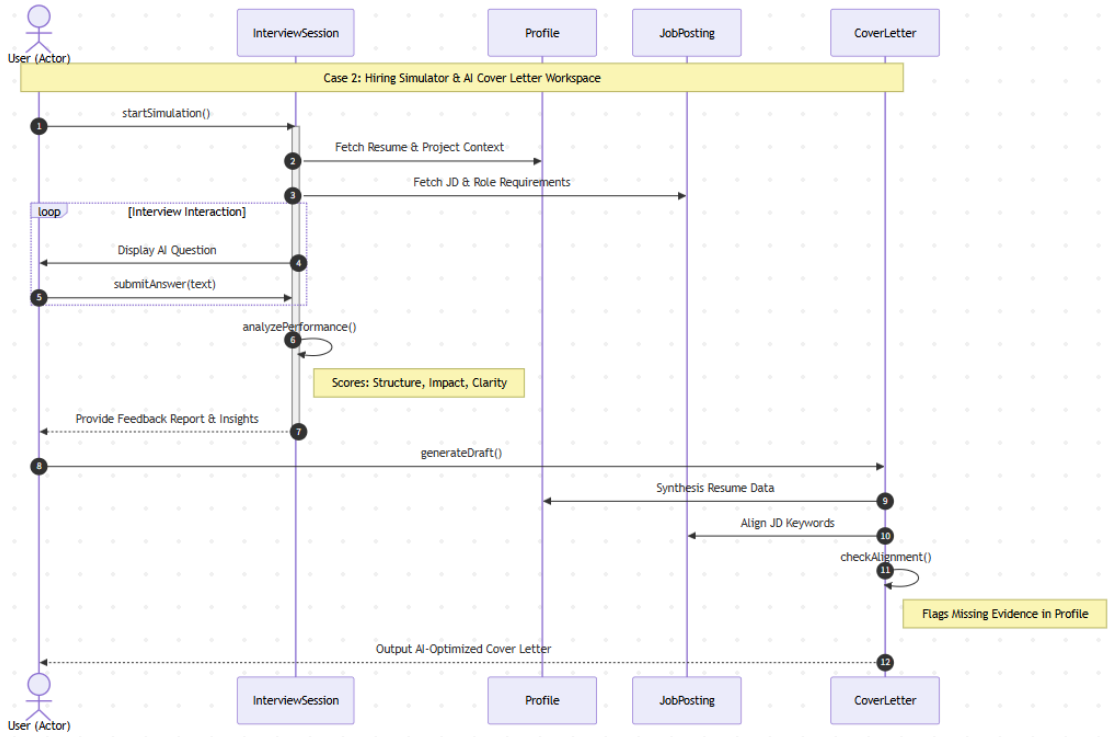
- ***fitScore* from AnalysisResult class**: A percentage score generated by AI that represents how well the user's profile aligns with a job posting.
- ***status* from SavedJob class**: Tracks where the user is in the application process—like 'saved', 'applying', 'interviewing', or 'completed'.
- ***readinessScore* from Dashboard class**: A real-time assessment score that represents the user's overall job market preparedness based on their total profile completeness and activity.
- ***matchLogic* from AnalysisResult class**: A detailed text-based explanation provided by the AI that describes the specific reasoning and rationale behind a job recommendation.
- ***scores* from InterviewSession class**: Multi-dimensional metrics (Structure, Impact, Clarity) stored in JSON format to provide granular feedback on a user's mock interview performance.
- ***missingEvidenceFlags* from CoverLetter class**: Automated alerts that identify specific requirements in a job description that are missing from the user's current profile or resume.
- ***keywordAlignment* from CoverLetter class**: A metric indicating how well the generated cover letter matches the essential keywords of a job description for ATS (Applicant Tracking System) optimization.
- ***workflowStage* from Dashboard class**: A visual state indicator that tracks a user's current progress through their personalized career roadmap.
- ***JSON* types** from projects, awards, etc..: These are stored as flexible JSON types so we can handle more complex or unstructured data (e.g., projects, roles, tech stack).

3.3 UML Sequence Diagram

Case 1: Dashboard Setup & Personalized Job Matching



Case 2: Hiring Simulator & AI Cover Letter Workspace



3.4 API Design

Category	Method	Endpoint URL	Input (Request Body)	Output (Response Body)	Description
Auth	POST	/api/auth/signup	{email, password}	{userSession, success}	User registration
Auth	POST	/api/auth/login	{email, password}	{authToken, user}	User authentication
Profile	POST	/api/profile/update	{major, gpa, skills[], resumeUrl}	{profileId, status}	Update user profile data
Dashboard	GET	/api/dashboard/stats		{readinessScore, weeklyTasks[]}	Fetch dashboard readiness and tasks
Matching	POST	/api/jobs/match	{jobId}	{fitScore, matchLogic, salaryIndex}	Analyze job fit using AI
Interview	POST	/api/interview/start	{jobId}	{sessionId, initialQuestion}	Initialize a mock interview session
Interview	POST	/api/interview/answer	{sessionId, answer}	{nextQuestion, realTimeFeedback}	Submit answer and receive next question
Interview	GET	/api/interview/report	{sessionId}	{totalScores, insightReport}	Retrieve final interview feedback report
Letter	POST	/api/write/generate	{jobId}	{content, missingEvidenceFlags}	Generate AI-powered cover letter
Tracking	PATCH	/api/jobs/status	{jobId, status}	{updatedStatus}	Update application tracking status

https://docs.google.com/spreadsheets/d/1jdC7KclbSgH-g5ZK_Q97vcSWbMoOt216kPUMrPtamA0/edit?gid=1110915406#gid=1110915406

3.5 Deployment

Our project combines **Vercel**'s automatic build and deployment pipeline with **Supabase**'s cloud database and authentication infrastructure to perform advanced AI analysis by linking **OpenAI API**s. In particular, Vercel and Supabase utilize the free plans of each service and solve the cost of using OpenAI through separate support to efficiently build and deploy the entire system infrastructure without incurring any cost.

3.6 Code Conventions

Complies with the Google JavaScript Style Guide for code readability and maintenance. The naming conventions apply camelCase (variable method), PascalCase (class), UPPER_SNAKE_CASE (constant) and are based on 2-space indentation, semi-colon usage, and lower quotation marks. It also automatically manages code style consistency among team members by introducing ESLint and Prettier.

4. Schedule

Milestone 2: Design Review (Completion: Early May)

Goal: Complete Software Design Document and determine system architecture.

Tasks:

- Software Design Document Finalization: Complete Document Structure and Text Organization (Heesang Kim)
- UML Diagram Precision: Class and Sequence Diagram Final Update (Dohyeon Lee)
- API Design Completed: Detailed API Specification with Google Sheet (Heesang Kim)

Milestone 3: Alpha Release (Completion: ~May 15)

Goal: Build basic infrastructure and complete User Interface framework.

Tasks:

- Core UI Layout: Career Cockpit Dashboard and Profile Management Page UI Development (Dohyeon Lee)
- Data API endpoints: Profile data CRUD and dashboard statistics deployment API (Heesang Kim)
- Common UI components: Create a button, input form library for reuse across the globe (Dohyeon Lee)

Milestone 4: Beta Release & Poster Presentation (Deadline: May 27)

Goal: Reinforcement of core AI features (simulator, matching) and deployment of website beta.

Tasks:

- AI Backend Logic: OpenAI API integration matching algorithm and interview question generation engine enhancement (Heesang Kim)
- Interactive AI workspace: Interview simulation interface and cover letter editing workspace check (Dohyeon Lee)
- Logic Optimization: AI Prompt Engineering Advancement and Matching Logic Exception Handling (Heesang Kim)
- Integration & Vercel Deployment: Final System Integration and Vercel Production Environment Deployment (Together)
- Poster presentation material production: Poster completion visualizing system architecture and core functions (Together)

Milestone 5: Final Release (Completion: ~June 15)

Goal: Reflects user feedback, optimizes performance, and final reports.

Tasks:

- Optimizing and Securing Performance: Improved API Response Speed, Enhanced Indexing and Data Security (Heesang Kim)
- UI/UX Polishing: Add animation, improve loading status, and final mobile responsive check (Dohyeon Lee)
- Reflects user feedback: Supplement functionality based on feedback collected at presentation (Together)
- Final Report: Documentation of project performance and future development directions (Together)

5. Contribution

Dohyeon Lee :

- Drafted the introduction and requirements sections for the Software Design Document.
- Served as the Lead Frontend Developer and UI Designer for the project.
- Developed the core user interface using Next.js and Tailwind CSS, including the Homepage, Profile management, and Dashboard.
- Designed and added new website design elements for the Hiring Simulator and AI Cover Letter Workspace to enhance user interaction.
- Implemented key functional requirements such as job filtering and real-time interview interfaces.
- Integrated high-fidelity dummy data to evaluate the system in a realistic service environment.
- Led the creation of high-fidelity mockups and UI flow diagrams to improve system visualization.

Heesang Kim :

- Drafted the system architecture sections for the Software Design Document.
- Served as the Lead Backend Architect and System Designer for the project.
- Designed the comprehensive database schema, entity relationships, and the detailed Data Dictionary.
- Defined the RESTful API specifications, including endpoint mapping and integration logic for OpenAI and Supabase.
- Designed UML Class and Sequence Diagrams to effectively model system behavior and data flow.
- Managed the GitHub repository, configured the Supabase infrastructure, and handled continuous deployment via Vercel.
- Finalized the technical design reports and updated the README with system setup and integration instructions.